Supplementary Material 2

1 Detailed Description of Predictive Models

1.1 Elastic Net (EN)

The Elastic Net (EN) model was employed to combine ℓ_1 (Lasso) and ℓ_2 (Ridge) penalties, simultaneously promoting variable selection and coefficient regularization. The minimized objective function is expressed as:

$$\min_{\mathbf{w}} \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha\rho\|\mathbf{w}\|_1 + \frac{\alpha(1-\rho)}{2}\|\mathbf{w}\|_2^2, \tag{1}$$

where $\alpha > 0$ controls the global intensity of regularization and $\rho \in [0, 1]$ defines the proportion between ℓ_1 and ℓ_2 penalties.

Implementation was performed with the `ElasticNetCV` class from the `scikit-learn` package, with automatic selection of hyperparameters by $k = 5$ fold cross-validation. The ρ parameter was evaluated on a predefined grid $(0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1.0)$, while α was tested across 100 values generated internally by the algorithm. The procedure was executed with a convergence tolerance of 10^{-4} and a maximum limit of 1,000 iterations.

1.2 LassoCV

The LassoCV model, a special case of Elastic Net with $\rho = 1$, was adjusted by minimizing the following function:

$$\min_{\mathbf{w}} \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_1. \tag{2}$$

The implementation used the `LassoCV` class from the `scikit-learn` package, which automatically selected the α hyperparameter by $k = 3$ fold cross-validation, exploring 100 candidate values. The maximum number of iterations was fixed at 10,000.

1.3 Support Vector Machines for Regression (SVR)

Support Vector Machines for Regression (SVR) were included to evaluate the capacity to capture non-linear relationships between genotypes and phenotypes. SVR seeks to find a function $f(\mathbf{x})$ whose predictions deviate from observed values $\mathbf{y}$ by at most an ϵ margin (*epsilon-insensitive loss*), penalizing only residuals whose magnitude exceeds this limit. The primal optimization problem was formulated as follows:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\xi}^*} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*), \tag{3}$$

subject to the constraints:

$$\begin{aligned} y_i - (\mathbf{w}^\top\phi(\mathbf{x}_i) + b) &\leq \epsilon + \xi_i, \\ (\mathbf{w}^\top\phi(\mathbf{x}_i) + b) - y_i &\leq \epsilon + \xi_i^*, \\ \xi_i, \xi_i^* &\geq 0, \end{aligned}$$

where $C > 0$ represents the regularization parameter, ϵ defines the width of the error-insensitive zone, and ξ_i, ξ_i^* are the slack variables. Through the *kernel trick*, the dual solution is expressed as:

$$f(\mathbf{x}) = \sum_{i=1}^{n} (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b, \tag{4}$$

where α_i and α_i^* denote the Lagrange multipliers and $K(\cdot, \cdot)$ is the kernel function.

SVR with Radial Basis Function (RBF) Kernel The SVR-RBF model utilized the kernel defined as:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\big(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\big). \tag{5}$$

Parameters followed the `scikit-learn` defaults: $\gamma = 1/p$ (where p is the number of markers).

SVR with Polynomial Kernel (Poly) The SVR-Poly model was adjusted with a polynomial kernel of degree $d = 3$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\, \mathbf{x}_i^\top \mathbf{x}_j + r)^d. \tag{6}$$

1.4 Random Forest (RF) and Extra Trees (ET)

RF and ET models were applied as ensemble methods based on *bagging* to mitigate variance. Both were configured with 200 decision trees. RF used subsampling with replacement, while ET used the complete dataset with random cut-off points. The splitting criterion was the Mean Square Error (MSE), with unlimited tree depth and a minimum of two samples required for node division.

1.5 Gradient Boosting Variants

Four boosting algorithms were employed to correct residuals sequentially:

- *HistGradientBoostingRegressor* (HGBR) was implemented as a computationally efficient evolution of traditional Gradient Boosting, utilizing a histogram-based binning technique for input features. This approach discretizes molecular marker values into 256 integer-valued bins, significantly reducing the complexity of finding optimal split points and allowing for efficient processing of large genomic matrices. The algorithm was configured via the `scikit-learn` library with a learning rate of 0.1, a maximum of 100 sequential iterations, and leaf-wise tree growth limited to 31 nodes. The Mean Squared Error (MSE) was adopted as the splitting criterion, incorporating early stopping to mitigate overfitting.

- *GradientBoostingRegressor* (GBR) was applied as a sequential ensemble method, where shallow decision trees are iteratively constructed to minimize the residuals (errors) of the preceding set. Unlike parallel ensemble methods, GBR employs a Gradient Descent optimization on the loss function to improve model convergence at each step. The implementation was performed using the `scikit-learn` library, configured with 100 estimators, a learning rate of 0.1, and a maximum tree depth of 3 to prevent overfitting. The Mean Squared Error (MSE) was used as the loss function, utilizing random sub-sampling to enhance the robustness of genomic predictions.

- The *LightGBMRegressor* (LGBM) was employed as a highly efficient gradient boosting framework that utilizes Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). Unlike traditional level-wise growth, LightGBM implements a leaf-wise (best-first) tree growth strategy, which focuses on reducing loss at specific nodes to achieve higher accuracy. The model was configured via the `lightgbm` library with 100 estimators,

a learning rate of 0.1, and 31 leaves per tree. No additional ℓ_1 or ℓ_2 regularization penalties were applied, following the default library settings, while maintaining the Mean Squared Error (MSE) as the primary objective function for genomic prediction.

- **XGBoost (XGB):** was implemented as an advanced gradient boosting framework optimized for speed and performance through parallel processing and tree pruning. Unlike standard boosting algorithms, XGBoost incorporates a regularized objective function that includes both ℓ_1 (Lasso) and ℓ_2 (Ridge) penalties directly into the loss function to control model complexity and prevent overfitting. The model was configured via the `xgboost` library with 100 estimators, a learning rate of 0.1, and a maximum tree depth of 6. Regarding regularization, the model followed the library's default parameters, where the ℓ_1 penalty (α) was set to 0 and the ℓ_2 penalty (λ) was set to 1, using Mean Squared Error (MSE) as the primary evaluation criterion.

1.6 K-Nearest Neighbors (KNN)

The *K-Nearest Neighbors* (KNN) model was implemented as a non-parametric regression algorithm. The predicted value $\hat{y}$ for a new individual $\mathbf{x}$ is the arithmetic mean of its K nearest neighbors in the SNP feature space:

$$\hat{y}(\mathbf{x}) = \frac{1}{K} \sum_{i \in \mathcal{N}_K(\mathbf{x})} y_i \tag{7}$$

where $\mathcal{N}_K(\mathbf{x})$ is the set of indices of the 10 closest training samples based on the Euclidean distance:

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{p} (x_j - x_{ij})^2} \tag{8}$$

The implementation used the `KNeighborsRegressor` class from `scikit-learn` with uniform weighting.